

۱ درونیابی لاگرانژ و اسپلاین مکعبی طبیعی

۱.۱ صورت مسئله

در این تمرین، مجموعه نقاط زیر در نظر گرفته شده است:

$$(1, 1), (2, 3), (3, 5), (4, 8), (5, 5), (6, 2).$$

هدف، به دست آوردن چندجمله‌ای درونیاب لاگرانژ برای این نقاط و همچنین محاسبه اسپلاین مکعبی طبیعی روی بازه‌های متوالی آن‌ها است. در روش لاگرانژ، یک چندجمله‌ای واحد از تمام نقاط عبور می‌کند؛ اما در روش اسپلاین، برای هر بازه یک چندجمله‌ای جداگانه ساخته می‌شود و این چندجمله‌ای‌ها در نقاط اتصال، شرایط همواری لازم را دارند.

۲.۱ درونیابی لاگرانژ

درونیابی لاگرانژ روشی برای ساختن چندجمله‌ای‌ای است که از تمام نقاط داده‌شده عبور کند. اگر نقاط داده‌شده به صورت زیر باشند:

$$(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n),$$

چندجمله‌ای درونیاب لاگرانژ به صورت رابطه (۱) تعریف می‌شود.

$$P(x) = \sum_{i=0}^n y_i L_i(x) \quad (1)$$

در این رابطه، $L_i(x)$ پایه لاگرانژ متناظر با نقطه i ام است و از رابطه (۲) به دست می‌آید.

$$L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}. \quad (2)$$

پایه لاگرانژ به گونه‌ای ساخته می‌شود که در نقطه x_i مقدار آن برابر ۱ و در سایر نقاط داده‌شده مقدار آن برابر صفر باشد. بنابراین در رابطه (۱)، هر جمله فقط مقدار تابع در نقطه مربوط به خود را در چندجمله‌ای نهایی اثر می‌دهد.

با جایگذاری نقاط داده شده در رابطه‌های (۱) و (۲)، چندجمله‌ای نهایی لاگرانژ به صورت رابطه (۳) به دست می‌آید.

$$P(x) = \frac{7x^5}{40} - \frac{71x^4}{24} + \frac{147x^3}{8} - \frac{1249x^2}{24} + \frac{1369x}{20} - 31 \quad (3)$$

این چندجمله‌ای از همه نقاط مسئله عبور می‌کند. برای نمونه داریم:

$$P(1) = 1, \quad P(2) = 3, \quad P(3) = 5, \quad P(4) = 8, \quad P(5) = 5, \quad P(6) = 2.$$

۱.۲.۱ برنامه محاسبه درونیابی لاگرانژ

در شکل ۱، کد مربوط به محاسبه چندجمله‌ای درونیاب لاگرانژ نمایش داده شده است.

```
import sympy as sp

x = sp.symbols('x')
xs = [1, 2, 3, 4, 5, 6]
ys = [1, 3, 5, 8, 5, 2]
n = len(xs) - 1

# ----- 1) Lagrange interpolation -----
P = 0
for i in range(len(xs)):
    Li = 1
    for j in range(len(xs)):
        if i != j:
            Li *= (x - xs[j]) / (xs[i] - xs[j])
    P += ys[i] * Li

P = sp.expand(P)
print("Lagrange polynomial:")
print(P)
```

شکل ۱: کد محاسبه درونیابی لاگرانژ

در این کد، ابتدا داده‌های ورودی در دو فهرست xs و ys ذخیره شده‌اند. سپس برای هر نقطه، پایه لاگرانژ مطابق رابطه (۲) ساخته شده و در مقدار تابع در همان نقطه ضرب شده است. در پایان، مجموع جمله‌ها طبق رابطه (۱) محاسبه، ساده و بسط داده شده است. محاسبات نمادین این بخش با استفاده از یک کتابخانه پایتونی انجام شده است.^۱ در شکل ۲، خروجی اجرای برنامه نمایش داده شده است.

Lagrange polynomial:

$7*x^{5/40} - 71*x^{4/24} + 147*x^{3/8} - 1249*x^{2/24} + 1369*x/20 - 31$

شکل ۲: خروجی محاسبه چندجمله‌ای لاگرانژ

همان‌طور که در شکل ۲ دیده می‌شود، برنامه پس از تشکیل پایه‌های لاگرانژ، چندجمله‌ای نهایی را ساده کرده و عبارت نوشته‌شده در رابطه (۳) را به دست می‌دهد.

۳.۱ اسپلین مکعبی طبیعی

در روش اسپلین مکعبی، به جای ساختن یک چندجمله‌ای واحد برای تمام نقاط، برای هر بازه بین دو نقطه متوالی یک چندجمله‌ای درجه سه ساخته می‌شود. در این مسئله ۶ نقطه وجود دارد؛ بنابراین ۵ بازه و در نتیجه ۵ چندجمله‌ای مکعبی خواهیم داشت. فرم کلی هر قطعه از اسپلین به صورت رابطه (۴) در نظر گرفته می‌شود.

$$S_i(x) = a_i x^3 + b_i x^2 + c_i x + d_i. \quad (4)$$

برای تعیین ضرایب مجهول، چند شرط روی قطعه‌های اسپلین اعمال می‌شود. نخست، هر قطعه باید از دو سر بازه خود عبور کند. همچنین در نقاط اتصال، مقدار تابع، مشتق اول و مشتق دوم باید پیوسته باشند. افزون بر این، چون اسپلین از نوع طبیعی است، مشتق دوم در دو سر کل بازه برابر صفر قرار داده می‌شود. شرط طبیعی بودن اسپلین در این مسئله به صورت رابطه (۵) نوشته می‌شود.

$$S''(1) = 0, \quad S''(6) = 0. \quad (5)$$

با اعمال این شرط‌ها، یک دستگاه معادلات برای ضرایب مجهول تشکیل می‌شود. حل این دستگاه، قطعه‌های زیر را برای اسپلین مکعبی طبیعی به دست می‌دهد:

SymPy^۱

$$S_0(x) = -\frac{39x^3}{209} + \frac{117x^2}{209} + \frac{340x}{209} - 1, \quad 1 \leq x \leq 2, \quad (6)$$

$$S_1(x) = \frac{195x^3}{209} - \frac{117x^2}{19} + \frac{3148x}{209} - \frac{2081}{209}, \quad 2 \leq x \leq 3, \quad (7)$$

$$S_2(x) = -\frac{28x^3}{11} + \frac{5256x^2}{209} - \frac{16481x}{209} + \frac{17548}{209}, \quad 3 \leq x \leq 4, \quad (8)$$

$$S_3(x) = \frac{470x^3}{209} - \frac{6768x^2}{209} + \frac{31615x}{209} - \frac{46580}{209}, \quad 4 \leq x \leq 5, \quad (9)$$

$$S_4(x) = -\frac{94x^3}{209} + \frac{1692x^2}{209} - \frac{10685x}{209} + \frac{23920}{209}, \quad 5 \leq x \leq 6. \quad (10)$$

بنابراین تابع اسپلاین به صورت تکه‌ای رابطه (۱۱) تعریف می‌شود.

$$S(x) = \begin{cases} S_0(x), & 1 \leq x \leq 2, \\ S_1(x), & 2 \leq x \leq 3, \\ S_2(x), & 3 \leq x \leq 4, \\ S_3(x), & 4 \leq x \leq 5, \\ S_4(x), & 5 \leq x \leq 6. \end{cases} \quad (11)$$

رابطه (۱۱) نشان می‌دهد که برخلاف روش لاگرانژ، در اینجا یک تابع تکه‌ای ساخته شده است. هر قطعه فقط روی بازه خودش معتبر است، اما همه قطعه‌ها در نقاط اتصال به صورت هموار به یکدیگر وصل می‌شوند.

۱.۳.۱ برنامه محاسبه اسپلاین مکعبی طبیعی

در شکل ۳، کد مربوط به محاسبه اسپلاین مکعبی طبیعی نمایش داده شده است.

```

# ----- 2) Natural cubic spline -----
#  $S_i(x) = a_i x^3 + b_i x^2 + c_i x + d_i$ 
a = sp.symbols(f'a0:{n}')
b = sp.symbols(f'b0:{n}')
c = sp.symbols(f'c0:{n}')
d = sp.symbols(f'd0:{n}')

eqs = []

for i in range(n):
    Si = a[i]*x**3 + b[i]*x**2 + c[i]*x + d[i]
    eqs.append(sp.Eq(Si.subs(x, xs[i]), ys[i]))
    eqs.append(sp.Eq(Si.subs(x, xs[i+1]), ys[i+1]))

for i in range(1, n):
    S_prev = a[i-1]*x**3 + b[i-1]*x**2 + c[i-1]*x + d[i-1]
    S_next = a[i]*x**3 + b[i]*x**2 + c[i]*x + d[i]

    eqs.append(sp.Eq(sp.diff(S_prev, x).subs(x, xs[i]),
                        sp.diff(S_next, x).subs(x, xs[i])))
    eqs.append(sp.Eq(sp.diff(S_prev, x, 2).subs(x, xs[i]),
                        sp.diff(S_next, x, 2).subs(x, xs[i])))

S0 = a[0]*x**3 + b[0]*x**2 + c[0]*x + d[0]
Sn = a[n-1]*x**3 + b[n-1]*x**2 + c[n-1]*x + d[n-1]
eqs.append(sp.Eq(sp.diff(S0, x, 2).subs(x, xs[0]), 0))
eqs.append(sp.Eq(sp.diff(Sn, x, 2).subs(x, xs[-1]), 0))

unknowns = list(a) + list(b) + list(c) + list(d)
sol = sp.solve(eqs, unknowns, dict=True)[0]

print("\nNatural cubic spline pieces:")
for i in range(n):
    Si = sp.expand((a[i]*x**3 + b[i]*x**2 + c[i]*x + d[i]).subs(sol))
    print(f"S_{i}(x) on [{xs[i]}, {xs[i+1]}] = {Si}")

```

شکل ۳: کد محاسبه اسپلاین مکعبی طبیعی

در این کد، برای هر بازه یک چندجمله‌ای مکعبی مطابق فرم کلی رابطه (۴) تعریف شده است. سپس شرط عبور هر قطعه از دو سر بازه خودش نوشته شده است. برای نقاط داخلی نیز پیوستگی مقدار تابع، مشتق اول و مشتق دوم اعمال شده است. در پایان، شرط طبیعی بودن

اسپلاین مطابق رابطه (۵) به دستگاه معادلات افزوده شده و دستگاه حاصل حل شده است. در شکل ۴، خروجی اجرای برنامه نمایش داده شده است.

```
Natural cubic spline pieces:
S_0(x) on [1, 2] = -39*x**3/209 + 117*x**2/209 + 340*x/209 - 1
S_1(x) on [2, 3] = 195*x**3/209 - 117*x**2/19 + 3148*x/209 - 2081/209
S_2(x) on [3, 4] = -28*x**3/11 + 5256*x**2/209 - 16481*x/209 + 17548/209
S_3(x) on [4, 5] = 470*x**3/209 - 6768*x**2/209 + 31615*x/209 - 46580/209
S_4(x) on [5, 6] = -94*x**3/209 + 1692*x**2/209 - 10685*x/209 + 23920/209
```

شکل ۴: خروجی محاسبه اسپلاین مکعبی طبیعی

خروجی شکل ۴ شامل ضرایب قطعه‌های مختلف اسپلاین است. این ضرایب همان قطعه‌هایی را می‌سازند که در رابطه‌های (۶) تا (۱۰) نوشته شده‌اند.

۴.۱ جمع‌بندی

در این تمرین، دو روش درونیایی برای مجموعه داده‌های مسئله بررسی شدند. روش لاگرانژ یک چندجمله‌ای واحد تولید کرد که از تمام نقاط داده‌شده عبور می‌کند. نتیجه این روش در رابطه (۳) نوشته شد.

در مقابل، روش اسپلاین مکعبی طبیعی یک تابع تکه‌ای ساخت که هر قطعه آن روی یک بازه مشخص تعریف می‌شود. این تابع علاوه بر عبور از نقاط داده‌شده، در نقاط اتصال نیز پیوستگی مقدار تابع، مشتق اول و مشتق دوم را حفظ می‌کند. همچنین با اعمال شرط طبیعی بودن، مشتق دوم در دو سر بازه اصلی برابر صفر قرار گرفت.

۲ تحلیل زمان اجرای الگوریتم‌ها و مصورسازی داده‌ها

۱.۲ صورت مسئله

در این بخش از پروژه، فایل داده‌ای با فرمت اکسل شامل زمان اجرای سه الگوریتم مختلف به ازای داده‌های ورودی با اندازه‌های متفاوت (۱۰۰ تا ۶۰۰ کیلوبایت) مورد بررسی قرار گرفته است. هدف، خواندن این فایل به صورت پویا با استفاده از زبان پایتون، رسم سه نوع نمودار (خطی، میله‌ای و جعبه‌ای)، محاسبه میانگین زمان اجرا برای یک الگوریتم خاص، و در نهایت بررسی قابلیت انعطاف‌پذیری کد در صورت افزوده شدن داده‌های جدید به فایل اکسل است.

۲.۲ خواندن فایل و رسم نمودارها (بدون داده ۷۰۰ کیلوبایت)

برای حل این بخش، از کتابخانه‌های پانداس^۲ جهت خواندن و پردازش داده‌ها و از متپلاتلیب^۳ جهت رسم نمودارها استفاده شده است. کد به گونه‌ای نوشته شده است که نام ستون‌ها و مقادیر آن‌ها را به صورت پویا پردازش کرده و عبارات اضافی مانند کیلوبایت^۴ را حذف می‌کند تا داده‌ها به اعداد قابل محاسبه تبدیل شوند. در شکل ۵، بخش مربوط به پردازش داده‌ها و کدهای رسم نمودارها در پایتون آورده شده است.

```
df.columns = df.columns.str.strip()

first_col = df.columns[0]

df[first_col] = df[first_col].astype(str).str.replace('KB', '', regex=False).str.strip()
df[first_col] = pd.to_numeric(df[first_col])

df.set_index(first_col, inplace=True)

#-----Plotting and saving as PDF-----

# --- Line Chart ---
ax1 = df.plot(kind='line', marker='o', figsize=(8, 5))
ax1.set_title('Algorithm Run Times')
ax1.set_ylabel('Run Time')
ax1.grid(True)
plt.tight_layout()
plt.savefig(current_dir / 'line_chart.pdf')

# --- Bar Chart ---
ax2 = df.plot(kind='bar', figsize=(8, 5))
ax2.set_title('Algorithm Run Times (Bar Chart)')
ax2.set_ylabel('Run Time')
ax2.grid(axis='y')
plt.tight_layout()
plt.savefig(current_dir / 'bar_chart.pdf')

# --- Box Plot ---
ax3 = df.plot(kind='box', figsize=(8, 5))
ax3.set_title('Run Time Distribution by Algorithm')
ax3.set_ylabel('Run Time')
ax3.grid(axis='y')
plt.tight_layout()
plt.savefig(current_dir / 'box_plot.pdf')
```

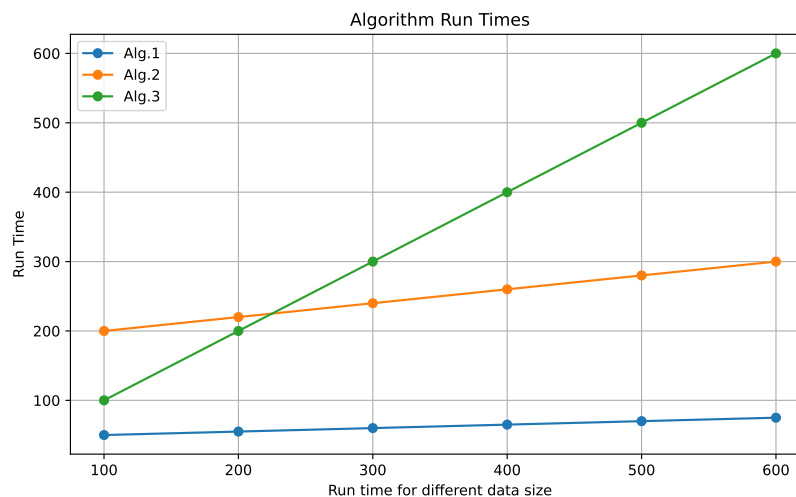
شکل ۵: کد پایتون برای پردازش داده‌های اکسل و رسم نمودارها

pandas^۲
matplotlib^۳
KB^۴

پس از اجرای کد بر روی داده‌های اولیه (۱۰۰ تا ۶۰۰ کیلوبایت)، سه نمودار زیر حاصل می‌شود.

۱.۲.۲ تحلیل نمودار خطی^۵

در شکل ۶ نمودار خطی زمان اجرای الگوریتم‌ها رسم شده است.



شکل ۶: نمودار خطی زمان اجرای الگوریتم‌ها (۱۰۰ تا ۶۰۰ کیلوبایت)

تحلیل: همان‌طور که از نمودار خطی پیداست، زمان اجرای الگوریتم ۳ با افزایش حجم داده‌ها با شیب تندی افزایش می‌یابد (از ۱۰۰ به ۶۰۰) که نشان‌دهنده حساسیت بالای این الگوریتم به حجم داده است. در مقابل، الگوریتم ۱ کمترین شیب و رشد را داشته و عملکرد بسیار پایداری دارد. الگوریتم ۲ با وجود اینکه در داده‌های کم (۱۰۰ کیلوبایت) بیشترین زمان اجرا را دارد، اما رشد آن ملایم‌تر از الگوریتم ۳ است و در حجم‌های بالاتر، از آن جا می‌ماند.

Line Chart^۵

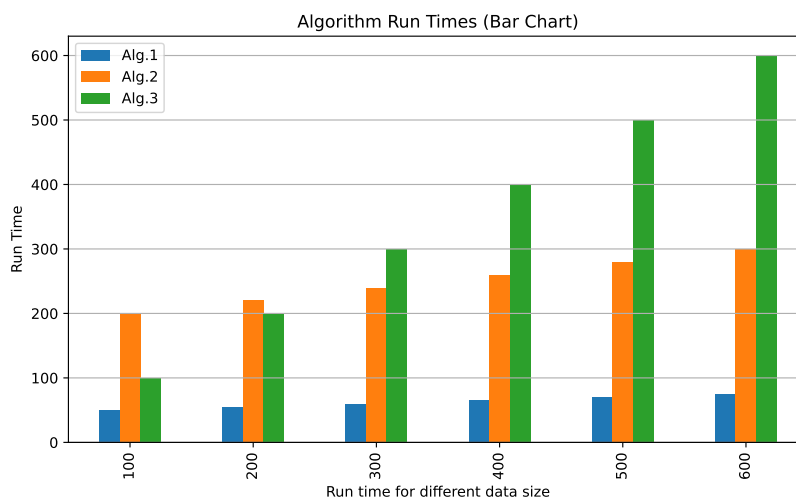
Alg.3^۶

Alg.1^۷

Alg.2^۸

۲.۲.۲ تحلیل نمودار میله‌ای^۹

در شکل ۷ نمودار میله‌ای داده‌ها قابل مشاهده است.



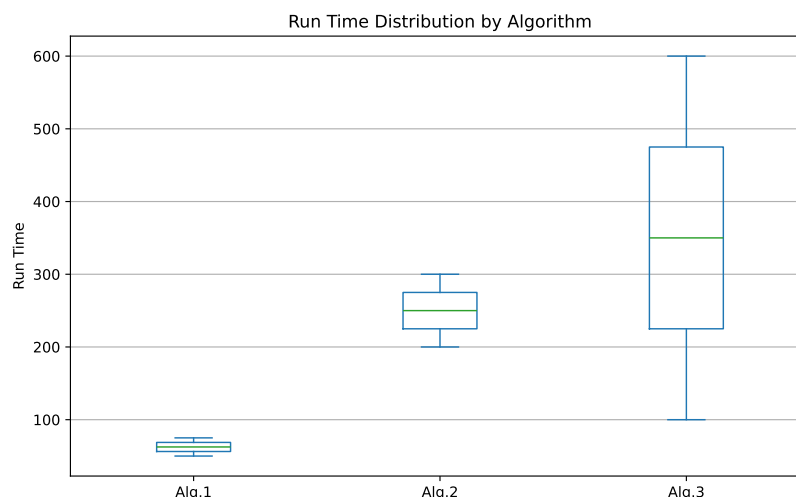
شکل ۷: نمودار میله‌ای مقایسه زمان اجرای الگوریتم‌ها در حجم داده‌های مختلف

تحلیل: نمودار میله‌ای به خوبی امکان مقایسه نظریه‌نظیر الگوریتم‌ها را در هر حجم داده فراهم می‌کند. کاملاً مشخص است که در حجم ۶۰۰ کیلوبایت، میله مربوط به الگوریتم ۳ بلندترین ارتفاع را دارد، در حالی که الگوریتم ۱ با زمان اجرای ۷۵، با اختلاف بسیار چشمگیری سریع‌تر از دو الگوریتم دیگر عمل کرده است.

۳.۲.۲ تحلیل نمودار جعبه‌ای^{۱۰}

در شکل ۸ نمودار جعبه‌ای برای بررسی توزیع داده‌ها رسم شده است.

Bar Chart^۹
Box Plot^{۱۰}



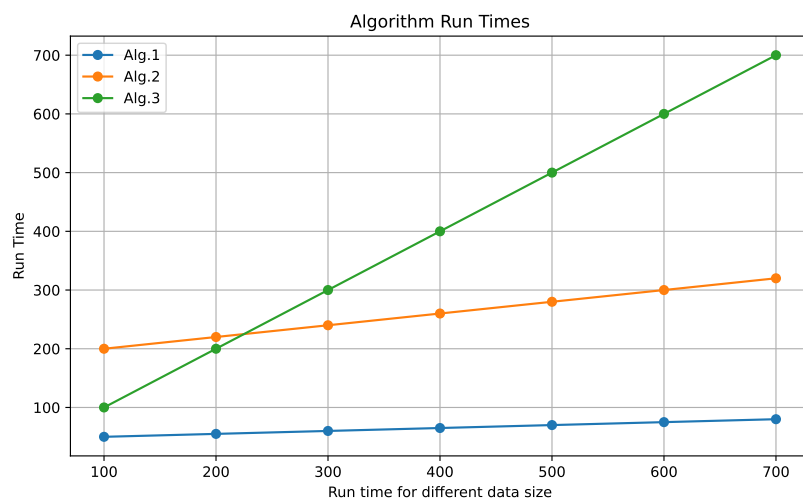
شکل ۸: نمودار جعبه‌ای توزیع زمان اجرای هر الگوریتم

تحلیل: نمودار جعبه‌ای میزان پراکندگی (واریانس) زمان اجرای هر الگوریتم را نشان می‌دهد. جعبه الگوریتم ۳ بسیار کشیده است که نشان می‌دهد زمان اجرای آن نوسان بالایی داشته و بازه گسترده‌ای را در بر می‌گیرد. در مقابل، جعبه الگوریتم ۱ بسیار فشرده است که نشانگر ثبات و تغییرات اندک این الگوریتم در مواجهه با افزایش اندازه داده ورودی است.

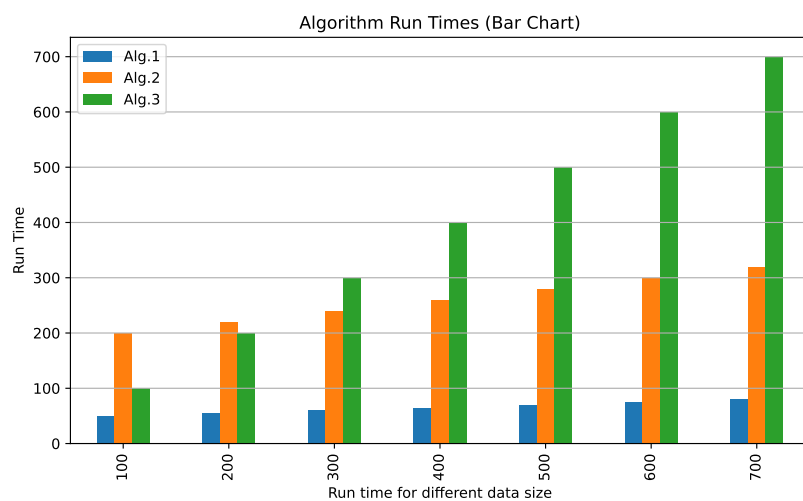
۳.۲ به‌روزرسانی داده‌ها (افزودن داده ۷۰۰ کیلوبایت)

طبق بند (ب) صورت سوال، ردیف جدیدی برای داده‌های با حجم ۷۰۰ کیلوبایت (با زمان‌های ۸۰، ۳۲۰ و ۷۰۰ برای الگوریتم‌های ۱ تا ۳) مستقیماً به فایل اکسل اضافه شد. به دلیل اینکه کد پایتون به صورت کاملاً پویا^{۱۱} نوشته شده است، بدون هیچ تغییری در سورس کد، برنامه مجدداً اجرا گردید و نمودارها به صورت خودکار با داده‌های جدید به‌روزرسانی شدند. نمودارهای به‌روزشده در شکل‌های ۹ تا ۱۱ آورده شده‌اند.

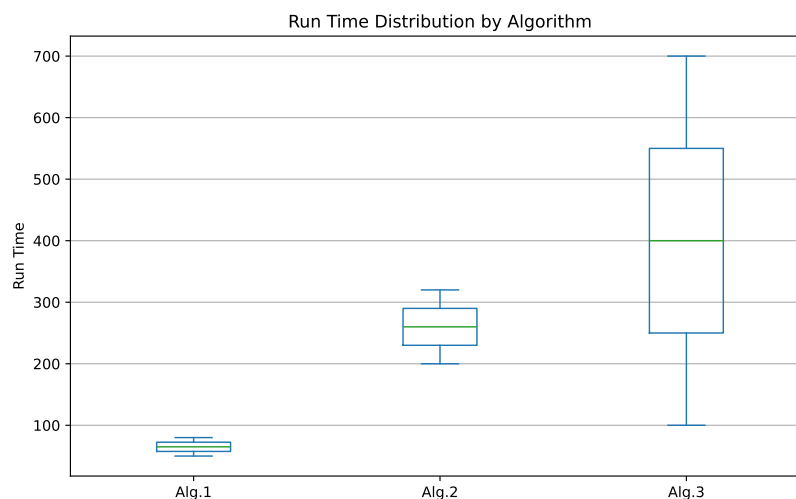
^{۱۱}Dynamic



شکل ۹: نمودار خطی زمان اجرا پس از افزودن داده ۷۰۰ کیلوبایت



شکل ۱۰: نمودار میله‌ای زمان اجرا پس از افزودن داده ۷۰۰ کیلوبایت



شکل ۱۱: نمودار جعبه‌ای توزیع زمان اجرا پس از افزودن داده ۷۰۰ کیلوبایت

تحلیل پس از افزودن داده‌ها: با اضافه شدن داده ۷۰۰ کیلوبایت، محور X در نمودارها به‌طور خودکار گسترش یافته است. نکته قابل توجه، ادامه روند صعودی شدید الگوریتم ۳ تا مقدار ۷۰۰ است که باعث شده در نمودار جعبه‌ای، کشیدگی جعبه این الگوریتم نسبت به قبل بیشتر هم بشود. الگوریتم ۱ همچنان با زمان اجرای ۸۰، بهترین و پایدارترین عملکرد را در حجم داده‌های بزرگ به نمایش می‌گذارد.

۴.۲ محاسبه میانگین زمان اجرای الگوریتم ۲

در بخش (ج) خواسته شده بود که میانگین زمان اجرای الگوریتم ۲ تنها برای داده‌های با حجم ۱۰۰ الی ۶۰۰ کیلوبایت محاسبه شود. کد این بخش با استفاده از فیلتر کردن ایندکس‌های دیتافریم بین ۱۰۰ و ۶۰۰ پیاده‌سازی شد.

در شکل ۱۲ کد مربوط به محاسبه میانگین آورده شده است.

```
# ----- Calculate Average for Alg.2 (100 to 600 KB) -----
if 'Alg.2' in df.columns:
    filtered_df = df.loc[(df.index >= 100) & (df.index <= 600)]
    avg_alg2 = filtered_df['Alg.2'].mean()
    print(f"Average run time for Alg.2 (100KB - 600KB): {avg_alg2:.2f}")
else:
    print("Column 'Alg.2' not found in the dataset.")
print("-" * 40)
```

شکل ۱۲: کد پایتون برای محاسبه میانگین الگوریتم ۲ در بازه مشخص

پس از اجرای کد، خروجی چاپ شده در ترمینال مطابق شکل ۱۳ به دست آمد.

```
Average run time for Alg.2 (100KB - 600KB): 250.00
```

شکل ۱۳: خروجی ترمینال و نمایش مقدار میانگین محاسبه شده

همان‌طور که در تصویر خروجی مشخص است، سیستم با موفقیت داده‌های ردیف ۱۰۰ تا ۶۰۰ را فیلتر کرده و داده ۷۰۰ کیلوبایتی (در صورت وجود در فایل اکسل) را در این محاسبه دخالت نداده است و مقدار دقیق میانگین چاپ شده است.

۳ محاسبه انتگرال $f(x) = e^{x^2}$

۱.۳ صورت مسئله

در این بخش، هدف محاسبه انتگرال معین تابع غیرخطی $f(x) = e^{x^2}$ در بازه $[0, 1]$ است. این تابع فاقد پادمشتق مقدماتی برحسب توابع ابتدایی است؛ از این رو برای محاسبه انتگرال آن از روش‌های تقریب عددی استفاده می‌شود. روش‌های پیاده‌سازی شده در این بخش شامل موارد زیر هستند:

۱. قاعده دوزنقه: تقسیم بازه به زیربازه‌های مساوی و تقریب زیر نمودار با استفاده از دوزنقه‌ها.

۲. قاعده سیمپسون: تقریب منحنی با استفاده از سهمی‌های عبوری از نقاط میانی هر دو زیربازه (تعداد زیربازه‌ها زوج در نظر گرفته می‌شود).

۳. انتگرال گیری گاوسی: استفاده از گره ها و وزن های بهینه سازی شده برای دستیابی به بیشترین دقت ممکن با کمترین تعداد ارزیابی تابع.

۲.۳ کد پایتون و پیاده سازی

پیاده سازی این روش ها در زبان پایتون با بهره گیری از کتابخانه های علمی انجام شده است. کد استفاده شده برای این محاسبات به همراه چاپ خروجی مستقیم برای هر روش در شکل ۱۴ نمایش داده شده است.

```
import numpy as np
from scipy.integrate import quad, simpson, trapezoid

a, b = 0, 1
n = 1000

x = np.linspace(a, b, n + 1)
y = np.exp(x**2)

trap_result = trapezoid(y, x)
print(f"Trapezoidal Rule: {trap_result:.15f}")

simp_result = simpson(y, x)
print(f"Simpson's Rule: {simp_result:.15f}")

gauss_result, error = quad(lambda x: np.exp(x**2), a, b)
print(f"Gaussian Quad: {gauss_result:.15f}")
```

شکل ۱۴: کد پایتون برای محاسبه انتگرال به روش های ذوزنقه، سیمپسون و انتگرال گیری گاوسی

۳.۳ نتایج و خروجی اجرای برنامه

پس از اجرای کد بالا بر روی بازه مشخص شده با تعداد زیربازه های ۱۰۰۰ ($n = 1000$)، نتایج عددی تا ۱۵ رقم اعشار به دست آمد. خروجی چاپ شده در محیط ترمینال در شکل ۱۵

نمایش داده شده است.

```
Trapezoidal Rule: 1.462652198954077
Simpson's Rule:   1.462651745907483
Gaussian Quad:    1.462651745907181
```

شکل ۱۵: خروجی اجرای کد و مقادیر انتگرال محاسبه شده

مقادیر استخراج شده از خروجی برنامه به شرح زیر است:

- قاعده ذوزنقه: ۱/۴۶۲۶۵۲۱۹۸۹۵۴۰۷۷
- قاعده سیمپسون: ۱/۴۶۲۶۵۱۷۴۵۹۰۷۴۸۳
- انتگرال گیری گاوسی: ۱/۴۶۲۶۵۱۷۴۵۹۰۷۱۸۱

۴.۳ تحلیل و مقایسه روش‌ها

با بررسی و مقایسه نتایج عددی به دست آمده از این سه روش، نکات زیر قابل تحلیل است:

۱. خطای روش ذوزنقه: روش ذوزنقه به دلیل استفاده از تقریب‌های خطی ساده در هر بازه، دارای بیشترین میزان خطا در مقایسه با سایر روش‌ها است. این موضوع خود را در تفاوت ارقام اعشار از مرتبه هفتم به بعد در مقایسه با روش گاوسی نشان می‌دهد.
۲. دقت روش سیمپسون: روش سیمپسون به دلیل استفاده از درونیایی درجه دو (سه‌موی) در هر دو زیربازه متوالی، به طرز چشمگیری دقیق‌تر از روش ذوزنقه است. مشاهده می‌شود که این روش تا ۱۲ رقم اعشار با نتیجه حاصل از انتگرال گیری گاوسی مطابقت دارد.
۳. کارایی و قدرت انتگرال گیری گاوسی: روش انتگرال گیری گاوسی پیاده‌سازی شده در کتابخانه علمی پایتون، به عنوان مرجع دقیق در نظر گرفته می‌شود. این روش با استفاده از انتخاب هوشمندانه نقاط ارزیابی (گره‌ها) و تخصیص وزن‌های بهینه به آن‌ها، بدون نیاز به افراز بازه به بخش‌های بسیار زیاد، دقیق‌ترین مقدار ممکن را برای انتگرال توابع هموار فرموله می‌کند.

۴ دسترسی به کدهای پروژه

تمامی کدهای برنامه پایتون، نمودارهای تولیدشده، فایل‌های داده و فایل گزارش این پروژه در یک مخزن عمومی در بستر گیت‌هاب بارگذاری شده و جهت ارزیابی در دسترس است. برای مشاهده و بررسی جزئیات کدهای پروژه، می‌توانید به آدرس زیر مراجعه فرمایید:

[https://github.com/rahimiparham2001/
mathematical-software-projects.git](https://github.com/rahimiparham2001/mathematical-software-projects.git)